

p No	Roll Number	Project Title	Project Description
1	R324ESA01, R324ESA03, R324ESA04	Race Condition Detection Tool	Build a system that detects race conditions in multithreaded applications by analyzing shared resource access patterns. The tool suggests synchronization solutions to improve program correctness.
2	R324ESA05, R324ESA06, R324ESA07	Thread Pool Management System	Design a thread pool framework that efficiently manages thread creation, reuse, synchronization, and termination. The system improves performance and scalability in concurrent server applications.
3	R324ESA08,R3 24ESA09, R324ESA10	Many-to-Many Thread Model Simulator	Create a simulator that demonstrates many-to-many thread mapping models and their impact on performance, scheduling efficiency, and system resource utilization.
4	R324ESA10, R324ESA11, R324ESA12	Lock-Free Synchronization Simulator	Develop a simulator that visualizes atomic operations and lock-free synchronization techniques. The project highlights high-performance concurrency methods used in modern multiprocessor systems.
5	R324ESA13, R324ESA14, R324ESA15	Secure File Management System	Develop a secure file management system implementing authentication, access control, encryption, and secure file operations. The system protects data confidentiality and integrity against unauthorized access.
6	R324ESA16, R324ESA17, R324ESA18	File System Recovery and Optimization Tool	Design a tool that simulates disk crashes and applies recovery mechanisms to restore data. The system also optimizes file read and write performance using efficient file system structures.
7	R324ESA19, R324ESA20, R324ESA21	Distributed File System with Fault Tolerance	Build a distributed file system that ensures data availability, consistency, and fault tolerance across multiple nodes. The system handles node failures gracefully while maintaining reliable data access.
8	R324ESA22, R324ESA23, R324ESA24	Journaling File System Simulator	Develop a simulator that demonstrates journaling techniques used in file systems. The project shows how journaling improves crash recovery and ensures data consistency.
9	R324ESA25, R324ESA26, R324ESA27	Disk Scheduling Algorithm Visualizer	Implement a visualization tool for disk scheduling algorithms such as FCFS, SSTF, SCAN, and C-SCAN. The simulator compares seek time, throughput, and performance efficiency.
10	R324ESA28, R324ESA29, R324ESA30	SSD Wear-Leveling Simulator	Create a simulator that demonstrates wear-leveling techniques in solid-state drives. The project explains how flash memory lifespan and reliability are improved through effective block management.
11	R324ESA31, R324ESA32, R324ESA33	OS Security Vulnerability Detection Framework	Develop a framework to detect operating system vulnerabilities such as buffer overflows, privilege escalation, and malware activity. The system provides alerts and mitigation recommendations.
12	R324ESA34, R324ESA35, R324ESA36	Secure Authentication Framework for OS	Design a secure authentication framework integrated with the operating system. The system supports multi-factor authentication, strong password policies, and protection against unauthorized access.
13	R324ESA37, R324ESA38, R324ESA39	System Call Monitoring and Auditing Tool	Create a monitoring tool that tracks system call usage and detects suspicious behavior. The system generates detailed audit logs to support security analysis and compliance.
14	R324ESA40, R324ESA41, R324ESA42	Real-Time Process Monitoring Dashboard	Develop a graphical dashboard that displays real-time information on process states, CPU utilization, memory usage, and overall system performance for administrators.
15	R324ESA43, R324ESA44, R324ESA45	Kernel-Level Intrusion Detection System	Design a kernel-level intrusion detection system that monitors abnormal behavior and detects security breaches in real time, enhancing operating system protection.
16	R324ESA46, R324ESA47, R324ESA48	Sandboxed Execution Environment	Develop a sandboxed execution environment that securely isolates untrusted programs. The system prevents unauthorized access to critical resources while allowing safe program execution.
17	R324ESA49, R324ESA50, R324ESA51	Intelligent CPU Scheduler Simulator	Develop an interactive CPU scheduling simulator implementing FCFS, SJF, Priority, and Round Robin algorithms. Users can define processes with arrival time, burst time, and priority while visualizing execution using Gantt charts and analyzing waiting time, turnaround time, and CPU utilization metrics.

18	R324ESA52, R324ESA53, R324ESA54	Energy-Efficient CPU Scheduling Algorithm	Design a power-aware CPU scheduling algorithm that reduces energy consumption using Dynamic Voltage and Frequency Scaling and thermal-aware scheduling. The system balances performance, fairness, and power efficiency, making it suitable for mobile, embedded, and battery-powered computing environments.
19	R324ESA55, R324ESA56, R324ESA57	AI-Based Predictive CPU Scheduler	Develop an intelligent CPU scheduler that uses machine learning techniques to predict process burst times based on historical execution patterns. The scheduler aims to reduce average waiting time, improve response time, and enhance overall system performance under varying workloads.
20	R324ESA58, R324ESA59, R324ESA60	Multi-Core Load Balancing Scheduler	Create a scheduler that dynamically distributes processes across multiple CPU cores to balance workload efficiently. The system monitors core utilization, prevents processor starvation, reduces load imbalance, and improves throughput and responsiveness in multi-core operating systems.
21	R324ESA61, R324ESA62, R324ESA63	Real-Time Scheduling Algorithm Simulator	Build a simulator for real-time scheduling algorithms such as Rate Monotonic and Earliest Deadline First. The tool visualizes task execution, deadlines, preemption, and missed deadlines, helping users understand real-time constraints and scheduling behavior.
22	R324ESA64, R324ESA65, R324ESA66	Priority Inversion Handling System	Design a system that demonstrates priority inversion problems in real-time systems and implements solutions such as priority inheritance and priority ceiling protocols. The project helps analyze scheduling anomalies and ensures predictable task execution.
23	R324ESA67, R324ESA68, R324ESA70	Process Lifecycle Visualization Tool	Develop a graphical visualization tool that illustrates the complete lifecycle of a process, including creation, ready, running, waiting, and terminated states. The system highlights context switching, scheduling decisions, and resource usage in an operating system.